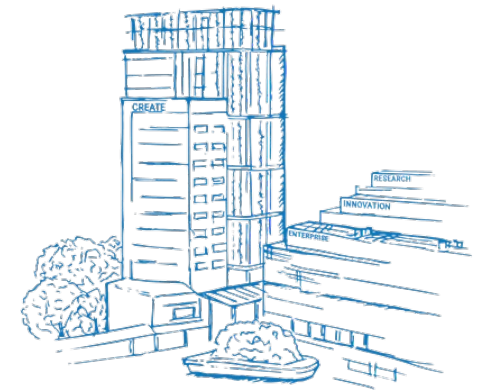


Agentic AI Workshop

Week 4 · Post-Training & Deployment. Why fine-tune, the post-training stages, full vs LoRA vs QLoRA, the data formats for tool-use and reasoning, and serving what you built.

Dr. Kerem Delikoyun · TUMCREATE · 17 June 2026



Where we are

Weeks 2–3. You used models *as they shipped*: prompts, tools, RAG, graphs. The LLM was a fixed black box.

Today. You change the model itself. We take a base model and shape its behavior with *your* data.

By the end you will have fine-tuned a model three ways and served it.

TODAY'S PATH

Prepare a dataset, then fine-tune three ways: full, LoRA, and QLoRA.

One dataset (MedQuAD), one small model (Qwen3-0.6B), start to finish.

Everything runs free on a Colab T4.

Why fine-tune at all?

A base model is a brilliant text-completer. It learned language from the open web, but it was never taught *your* format, *your* domain voice, or when to stop talking.

Three ways to adapt a model, cheapest first:

- **Prompt it** , no training, just change the instructions
- **Give it documents** , RAG, retrieve facts at query time
- **Change the weights** , fine-tuning, bake the behavior in

REACH FOR FINE-TUNING WHEN

You need a **consistent format** every time (our JSON answers).

A **domain voice** the base model does not have.

A **small, private model** you run on your own hardware.

Rule of thumb: fine-tuning reshapes *how* a model responds more than *what* it knows. For fresh facts, prefer RAG or continued pre-training.

Pre-training builds it, post-training makes it usable

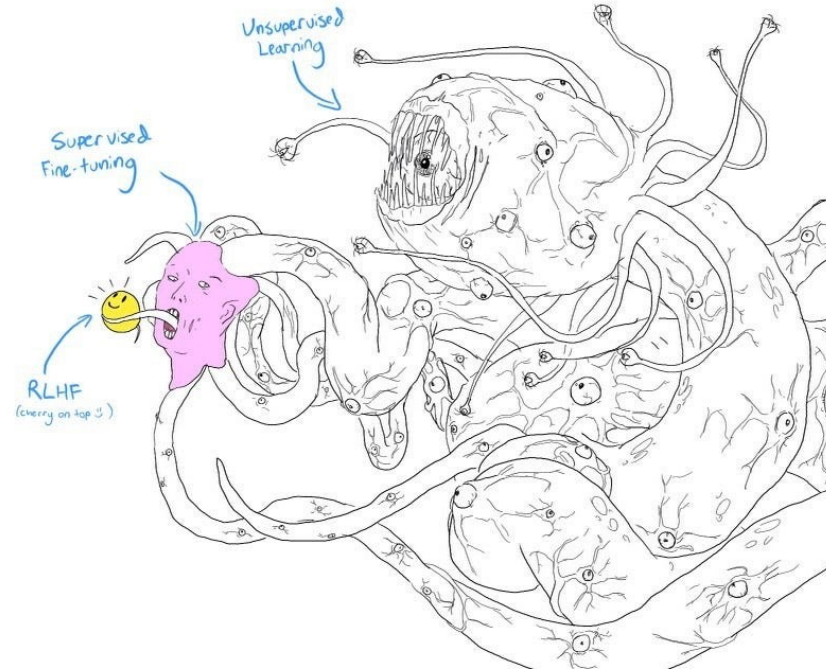
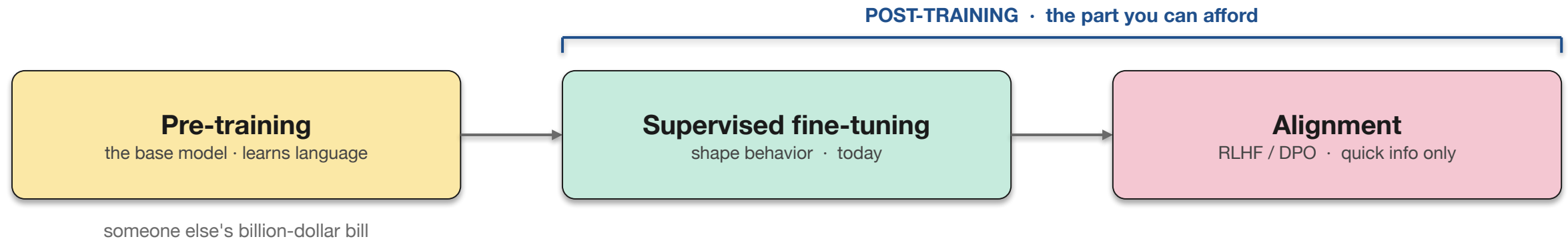


Image: Chip Huyen, RLHF (huyenchip.com/2023/05/02/rlhf.html)

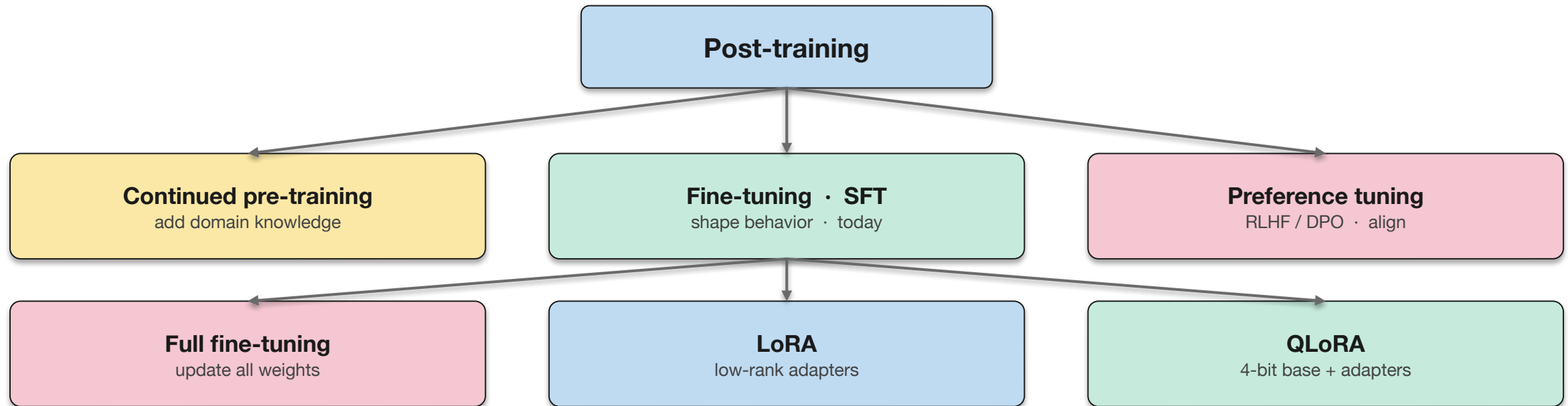
The Shoggoth: pre-training spends almost all the compute and gives the model raw intelligence with no manners. Fine-tuning is the small friendly mask, and RLHF is the cherry on top. We work in that thin outer layer.

The LLM training pipeline



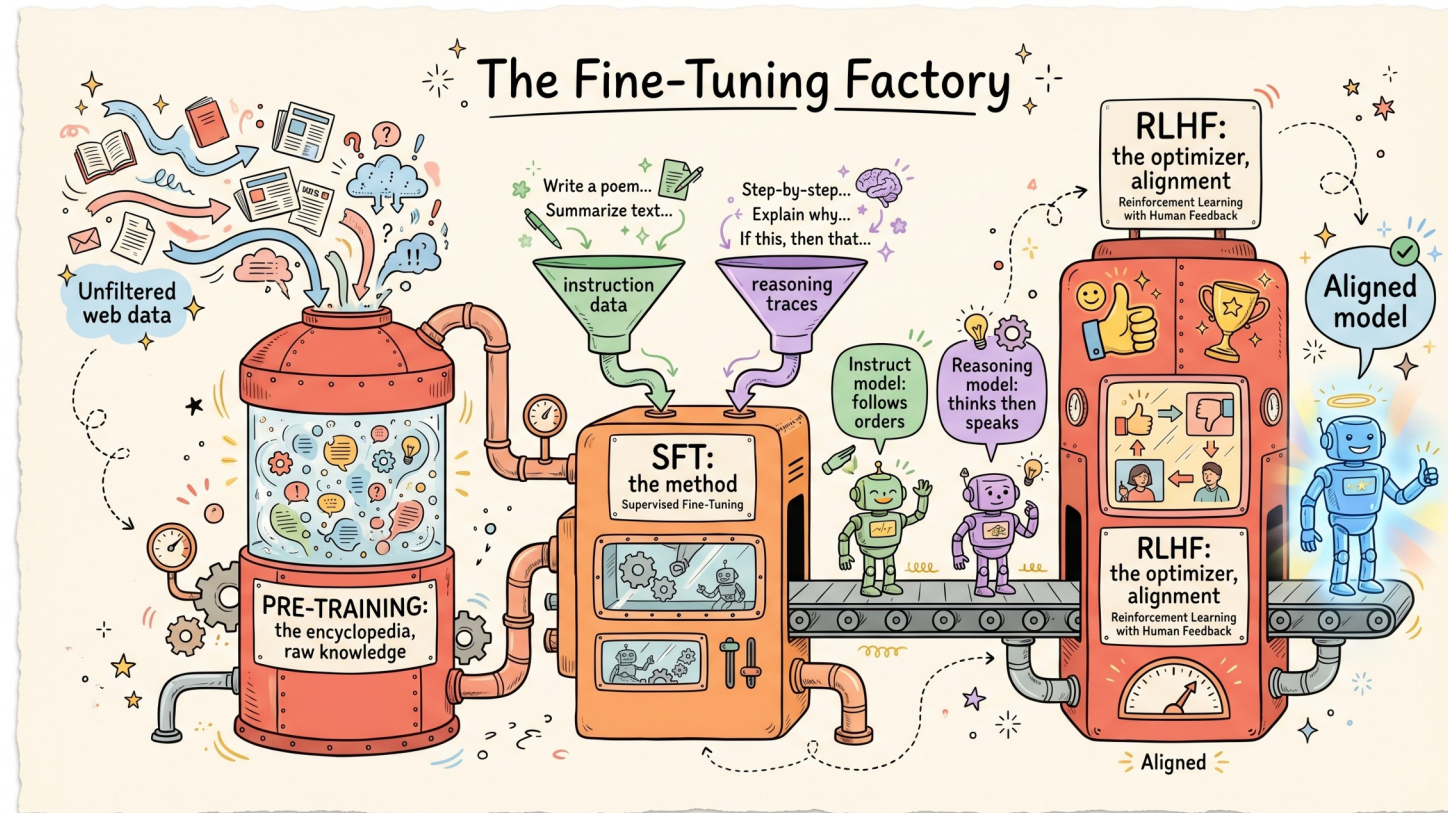
Pre-training learns language from the open web, that is the base model, and someone else paid for it. Post-training is the part you can afford: it shapes behavior. OpenAI's InstructGPT made this recipe the standard in 2022.

Post-training, and where today fits



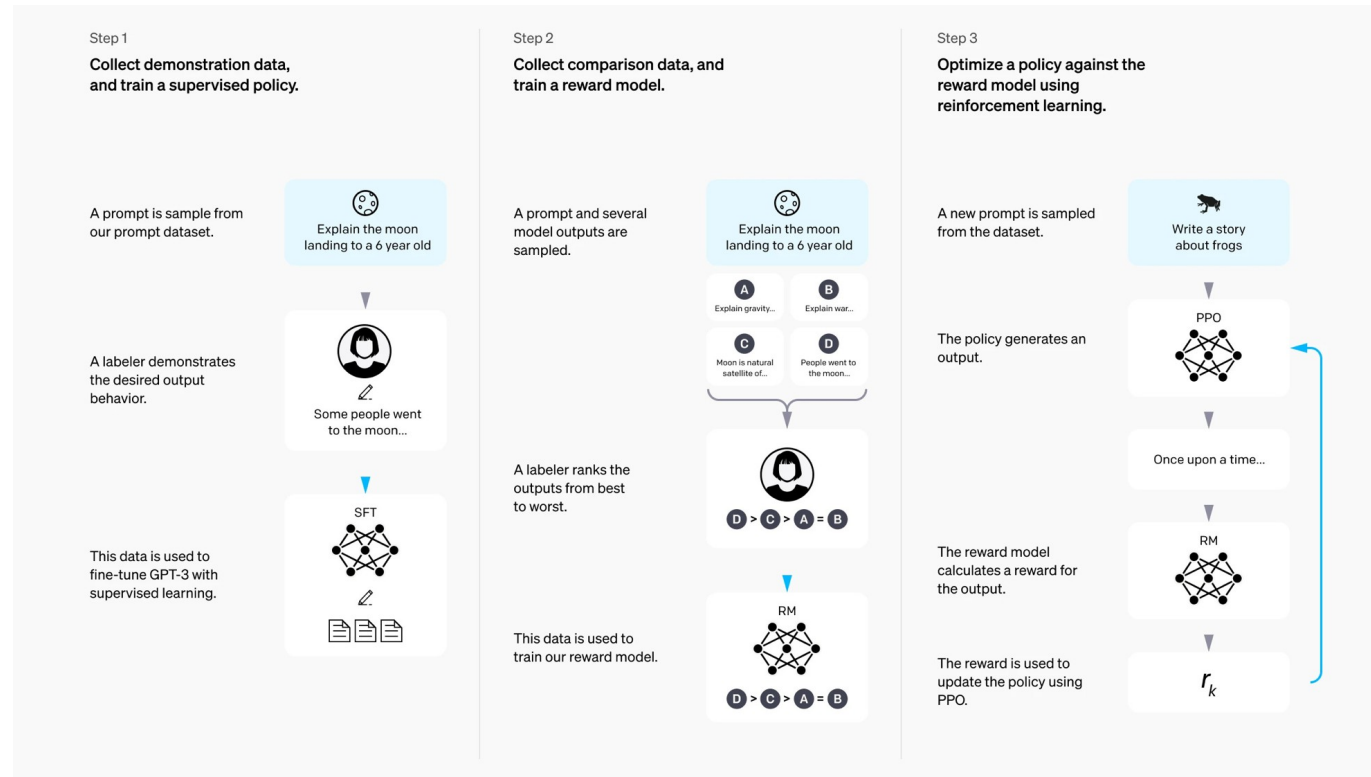
Continued pre-training adds domain knowledge. Preference tuning (RLHF or DPO) aligns the model to human taste, we cover it on slides only. Today is supervised fine-tuning, and we do it three ways: full, LoRA, QLoRA.

The fine-tuning factory



One picture for the whole journey: raw web data becomes a base model in pre-training, supervised fine-tuning shapes its behavior, and alignment polishes it. Today we run the middle stage.

Alignment, in three steps



SFT and RLHF steps. Image: OpenAI, Aligning language models to follow instructions (openai.com/index/instruction-following)

After SFT, alignment teaches *taste*: collect human rankings, train a reward model, then optimize the policy against it with PPO. We do not run this today, but it is the rung above fine-tuning, and it is how ChatGPT was made.

PART 1 · NOTEBOOK 00

The dataset is the project

Everyone wants to have a good model, but the dataset is 90% of the result.

MedQuAD, and the labels it already ships

MedQuAD = ~47k question/answer pairs from 12 NIH/NLM sites, each answer clinician-written.

It comes with **weak structured labels**:

- `question_focus`: the medical entity
- `question_type`: symptoms / treatment / causes ...
- `umls_semantic_group`: the concept category

Real datasets arrive with structure you can exploit, and later use as partial ground truth.

ONE RAW ROW

Q: What are the symptoms of Hashimoto's disease?

focus: Hashimoto's Disease

type: symptoms

A: *"Hashimoto's disease often causes fatigue, weight gain, cold intolerance..."*

Teacher → student labeling

A **strong model labels data a small model learns to imitate**. This is how Alpaca, OpenOrca, and most instruction sets were actually built.

Here: **Gemini** reads each Q&A and extracts a tight clinical schema as **structured JSON**, guaranteed parseable, no regex.

```
disease      : "Hashimoto's disease"
patient_info : "middle-aged women"
symptoms     : [fatigue, weight gain, ...]
treatment    : [levothyroxine]
answer_summary : "...autoimmune thyroiditis..."
```

THE DECISION THAT MAKES EVALUATION POSSIBLE

We make the **answer a JSON object**, not free prose.

That turns a fuzzy *"is this a good answer?"* into a measurable *"did it get the disease / symptoms / treatment right?"*

Schema design is eval design.

Pack it into the chat template

The example is a **conversation**, rendered with the model's *own* template so the special tokens match exactly.

```
<|im_start|>system
You are a clinical assistant. Respond with JSON: disease, patient_info,
symptoms, treatment, answer_summary.<|im_end|>
<|im_start|>user
What are the symptoms and treatment of Hashimoto's disease?<|im_end|>
<|im_start|>assistant
{"disease": "Hashimoto's disease", "symptoms": ["fatigue", ...], ...}<|im_end|>
```

We score the loss only on the **assistant** tokens. The user's words are context, not a target, so the model learns to *respond*, not echo the question back.

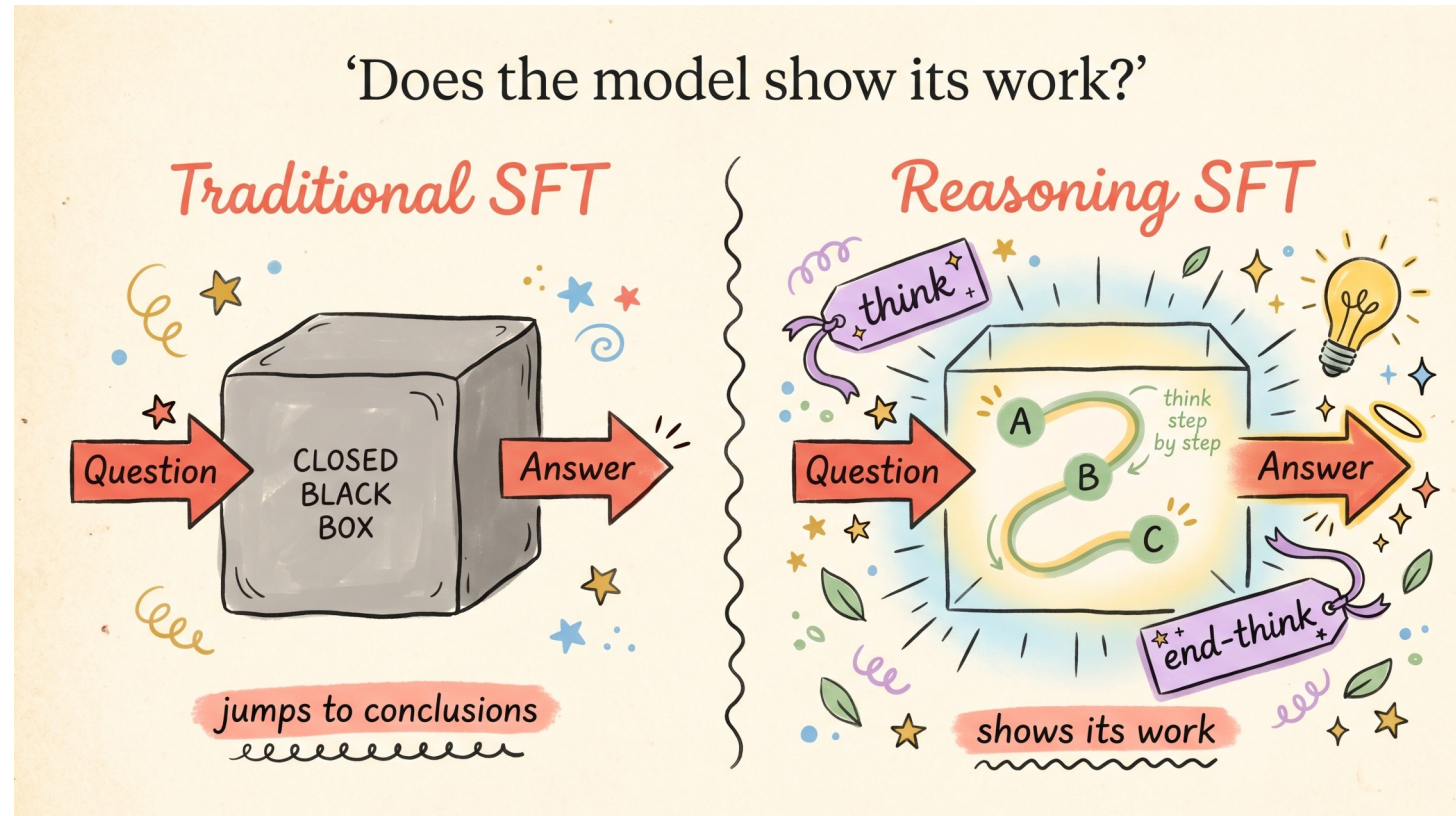
THE TEMPLATE IS A CONTRACT

Those `<|im_start|>` tags mark **structural zones**: this is the user, this is the assistant, this is a tool call. It lives in `tokenizer_config.json` as a Jinja2 string.

MATCH IT EXACTLY

Fine-tune with one template and deploy with another, and the model **breaks**: it answers for the user, or it never stops.

SFT teaches reasoning, not just answers



Plain SFT learns the jump from question to answer and invites shortcutting, where hallucinations live. Reasoning SFT trains the whole path with a <think> trace. DeepSeek-R1 showed you must seed this with SFT first, then let RL refine it.

What a reasoning example looks like

USER

```
<|im_start|>user  
What is 17 x 23?<|im_end|>
```

ASSISTANT · reasoning trace

```
<|im_start|>assistant  
<think>  
17 x 23 = 17x20 + 17x3  
        = 340 + 51  
</think>
```

ASSISTANT · final answer

```
391<|im_end|>
```

THE STRUCTURE IS THE LESSON

It is one **assistant turn** with two zones: a `<think>` block, then the answer.

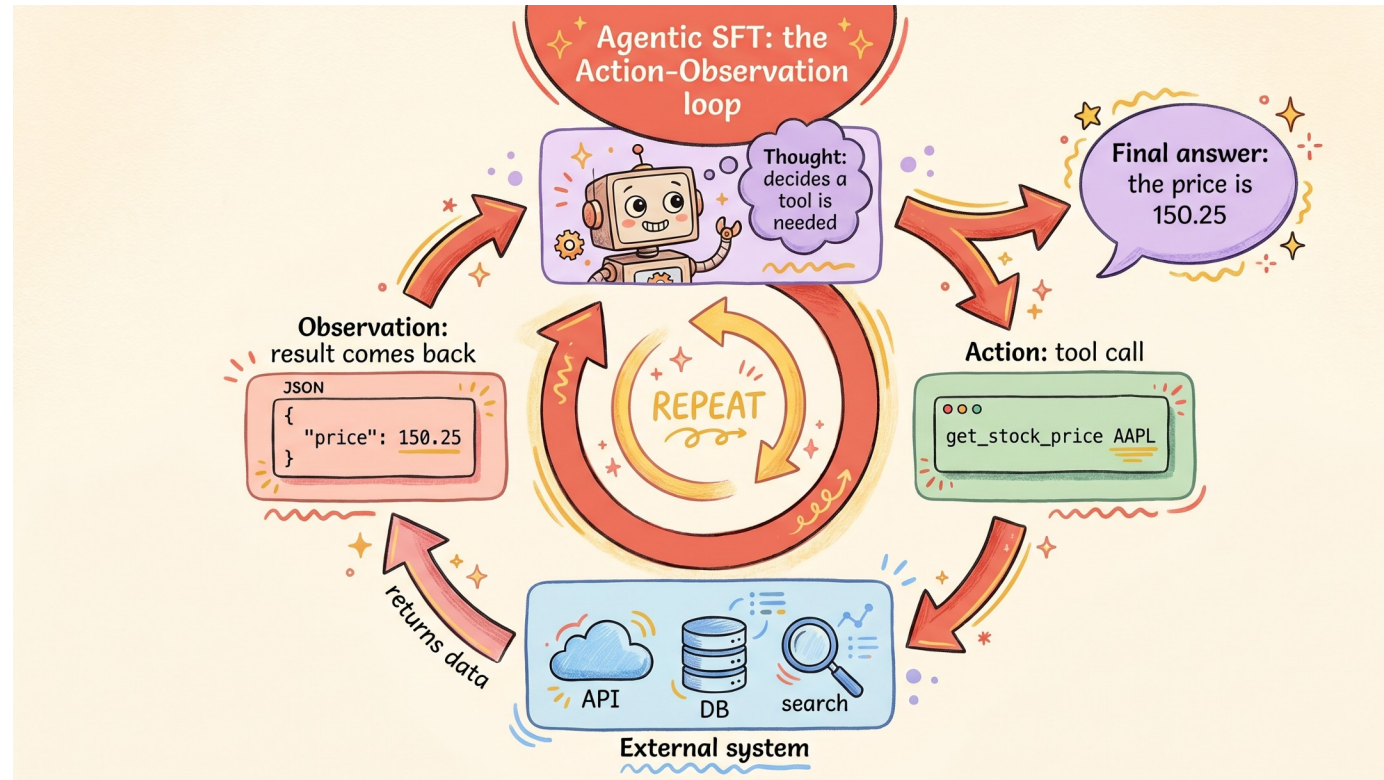
On Qwen3 you fill a `reasoning_content` field and the template renders the `<think>` tags for you.

Train with the trace and the model learns to *work things out* first. Train without it and it learns to *guess*.

Chat templates: (huggingface.co/docs/transformers/chat_templating).

Qwen3 chat template (<https://huggingface.co/blog/qwen-3-chat-template-deep-dive>)

Agentic SFT: teaching the model to use tools



The most on-theme step for an agents course: the model learns to emit a precise tool call, pause, read the result, and continue. Format is non-negotiable, a missing bracket and the call never executes. Training data is clean, often synthetic, tool-calling traces.

What a tool-use example looks like

SYSTEM · tools declared

```
<tools>
  {"name": "get_stock_price", ...}
</tools>
```

USER

What is Apple's stock price?

ASSISTANT · tool call

```
<tool_call>
  {"name": "get_stock_price",
   "arguments": {"ticker": "AAPL"}}
</tool_call>
```

TOOL · response

```
<tool_response>
  {"price": 150.25}
</tool_response>
```

ASSISTANT · final answer

Apple is trading at \$150.25.

TEXT WITH ARCHITECTURAL ZONES

Once you add `<tool_call>` and `<tool_response>` tokens, the conversation is no longer plain text. Each zone has a **role**: user intent, the assistant's voice, an API interaction.

The model pauses after the tool call, the system runs the tool, the result is fed back, and only then does it answer.

Token style varies by family (`<tool_call>` on Qwen, `<|tool_call|>` on others). Match your model's exactly.

Format: Qwen3 chat template.

The library doing the work: Unsloth + TRL

- Every training notebook this week runs on **Unsloth**, a drop-in faster trainer
- **2x faster, about half the VRAM**, same Hugging Face models and weights
- One API:
`FastLanguageModel.from_pretrained(...)`, then flip a flag
- **TRL's `SFTTrainer`** runs the actual training loop on top

SAME NOTEBOOK, ONE LINE CHANGES

full SFT ·

```
from_pretrained(full_finetuning=True)
```

LoRA · `load_in_4bit=False +`
`get_peft_model(r=32)`

QLoRA · `load_in_4bit=True +`
`get_peft_model(r=32)`

```
from unsloth import FastLanguageModel
model, tok = FastLanguageModel.from_pretrained("Qwen/Qwen3-0.6B", load_in_4bit=False)
# LoRA / QLoRA add one line: model = FastLanguageModel.get_peft_model(model, r=32, lora_alpha=16)
from trl import SFTTrainer
trainer = SFTTrainer(model, tokenizer=tok, train_dataset=ds, args=cfg) # TRL runs the loop
```

Why it matters: full into LoRA into QLoRA is the same notebook with one flag flipped, and Unsloth makes all three fit on a free Colab GPU.

PART 2 · NOTEBOOK 01

Full fine-tuning

The honest baseline: update every weight. Best quality, heaviest bill, and the reason LoRA and QLoRA exist.

Full fine-tuning: update every weight

Update **100% of the weights**. This is the honest baseline and the best quality ceiling.

`full_finetuning=True`, no quantization. Use a low learning rate (**$2e-5$**): nudge every weight gently, or you risk **catastrophic forgetting**, overwriting general skills while learning a narrow task.

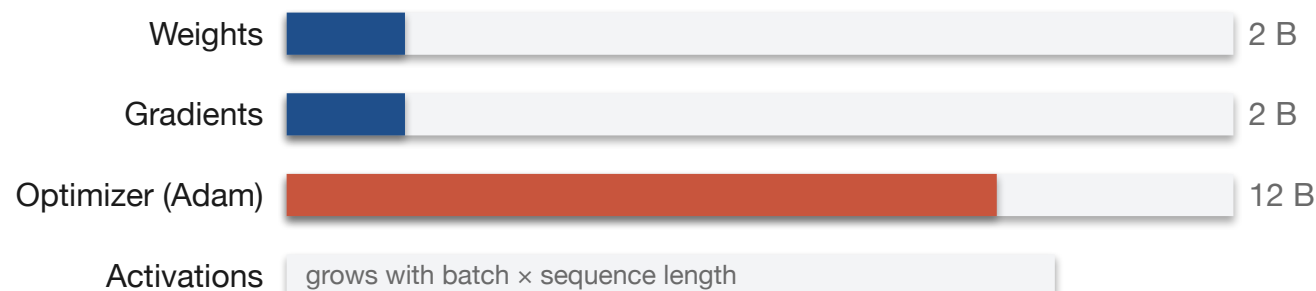
Saved artifact is the whole model: ~**1.2 GB** for Qwen3-0.6B, and far larger for a real model.

WHY IT GETS EXPENSIVE

For every weight you train, the optimizer also stores a **gradient** and **two Adam states**.

That is the memory wall the next two methods climb over.

Where the memory goes



≈ 16 bytes / parameter

A 7B model is ~112 GB before activations. That is the wall.

Full fine-tuning pays all of this for every weight. LoRA pays the optimizer and gradient buckets for the tiny adapters only. QLoRA also shrinks the weights bucket to 4-bit. That is the whole story of the next two methods.

PART 3 · NOTEBOOK 02

LoRA

The trick that made fine-tuning democratic: freeze the model, train a tiny correction beside it.

Why not just full fine-tune?

PROBLEM 1: THE VRAM WALL

A gradient plus two Adam states for every one of billions of weights.

Full fine-tuning a 70B model needs **over a terabyte** of VRAM before you feed it a single token.

PROBLEM 2: CATASTROPHIC FORGETTING

Full freedom in weight space lets gradients **overwrite** useful general knowledge.

The model gets better at the narrow task and worse at everything else.

LoRA answers both: freeze the base, and train a tiny low-rank correction beside it.

LoRA: the matrix shapes

$$\text{Output} = W \cdot X + (B \cdot A) \cdot X \quad W \text{ frozen} \cdot \text{only A and B train}$$

- **W** (frozen) has shape $d \times d$: a full update ΔW would too
- LoRA factors the update: $\Delta W = B \cdot A$, with a skinny inner rank $r \ll d$
- **A** is $r \times d$ ($d \rightarrow r$), **B** is $d \times r$ ($r \rightarrow d$)
- Multiply back: $(d \times r) \cdot (r \times d) \rightarrow d \times d$, same shape as **W**, so $W + BA$ is valid

WHERE THE SAVING COMES FROM

full ΔW : d^2 trainable params

LoRA: $2 \cdot d \cdot r$ trainable params

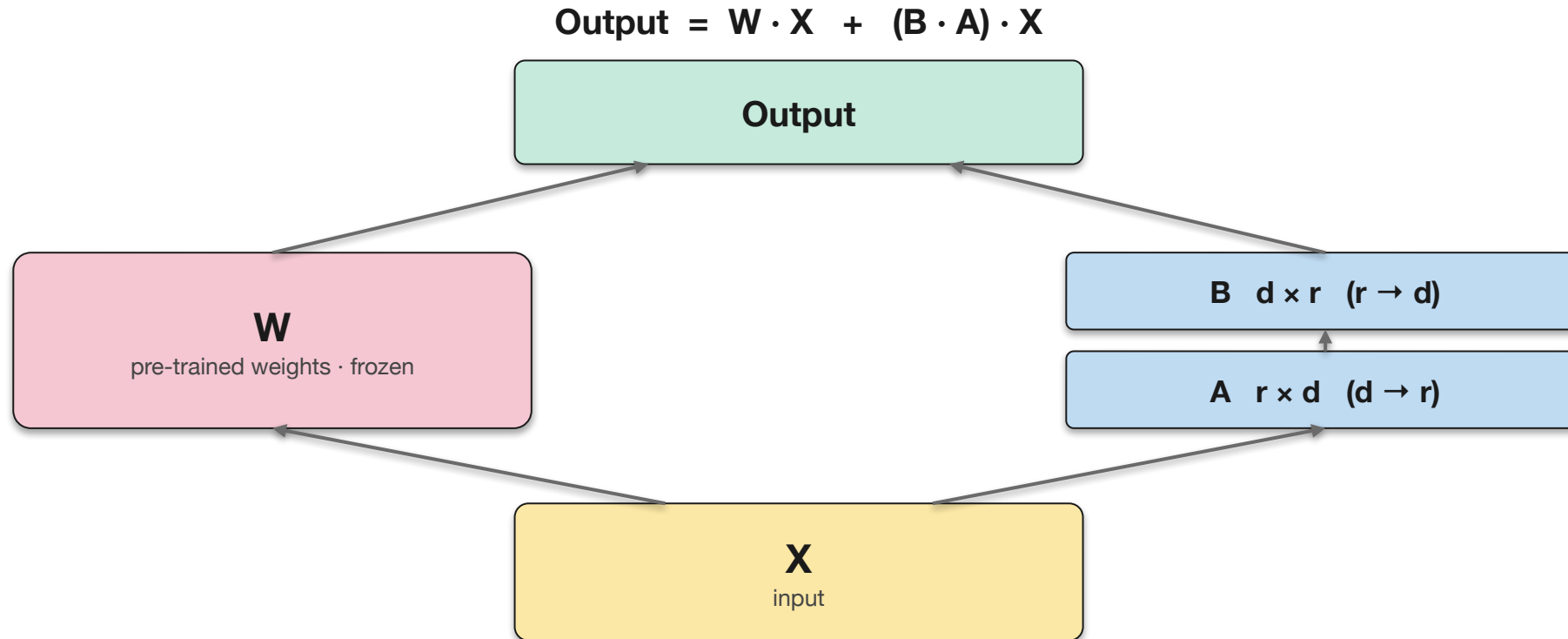
$d = 4096, r = 8 \rightarrow$ **16.8M vs 65.5K**

$\approx$ **256x fewer** params to train

matmul rule: $(m \times k) \cdot (k \times n) \rightarrow (m \times n)$ # the shared inner dim must match
 LoRA update: $B(d \times r) \cdot A(r \times d) \rightarrow (d \times d)$ # inner rank r is the bottleneck
 forward: $A(r \times d) \cdot X(d \times n) \rightarrow (r \times n) \rightarrow B(d \times r) \cdot (r \times n) \rightarrow (d \times n)$

Only A and B receive gradients, so optimizer and gradient memory scale with $2dr$, not d^2 . That is the whole memory win behind LoRA and QLoRA.

LoRA: train a low-rank correction



Freeze W . Learn the update as $B \cdot A$ with skinny A and B (rank r). B starts at 0, so you begin exactly as the base model.

The intrinsic-rank hypothesis: the *update* a task needs lives in a tiny subspace, even when W is enormous. So instead of a full dense change, learn $\Delta W = B \cdot A$ with skinny A and B . Only A and B get gradients, so the optimizer and gradient memory shrink $\sim 100\times$.

The LoRA knobs that matter

knob	what it controls	typical
Rank (r)	size of the correction subspace (its capacity)	8 to 32
Alpha (α)	strength of the update, applied as α / r	r or $2r$
target_modules	which layers get adapters	q/k/v/o_proj, MLP
learning rate	optimization speed (base is frozen, so higher)	$\sim 2e-4$

Trainable parameters drop to ~ 1 to 2%. The saved adapter is **MB**, not **GB**, so you can email it, and swap many task adapters over one shared base model.

PART 4 · NOTEBOOK 03

QLoRA

LoRA shrank what we train. QLoRA shrinks what we store, so one consumer GPU can fine-tune a frontier-size model.

QLoRA: the last wall

LoRA shrinks what you *train*. But the frozen base still has to **live in memory**, and in 16-bit a 7B model is **14 GB** before you process a single token.

QLoRA's move: store the frozen base in **4-bit**, so it shrinks about **4x**.

What you train stays small. What you store gets small too.

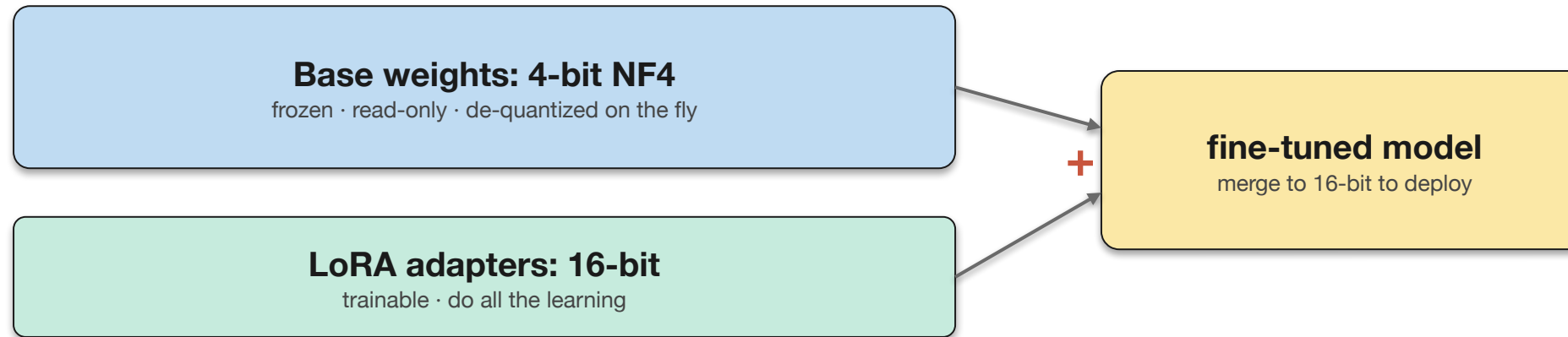
THREE TRICKS MAKE IT WORK

NF4, a 4-bit number type shaped to the weight bell-curve, so its 16 levels land where weights actually cluster.

Double quantization, quantize the quantization constants too, for a little more.

Paged optimizers, spill to CPU RAM on a memory spike instead of crashing.

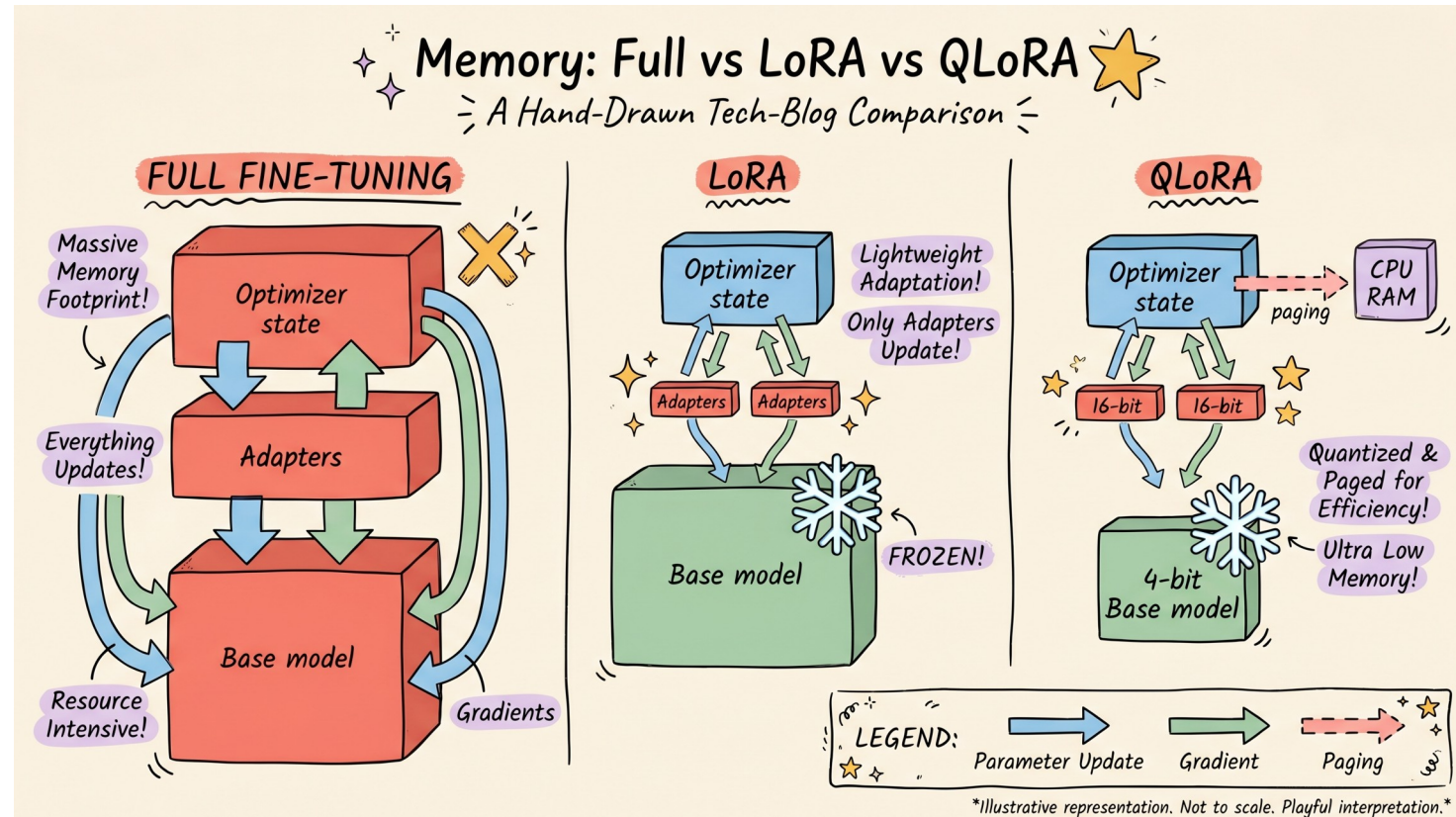
QLoRA: quantize the frozen base



16-bit to 4-bit: the frozen base shrinks about 4x (a 7B model goes from ~14 GB to ~4 GB)

The 4-bit base is read-only and de-quantized on the fly, the 16-bit adapters do all the learning. This is what lets one consumer GPU fine-tune a 7B model. Merge the adapter back to 16-bit at the end for easy deployment.

Full, LoRA, QLoRA: where the memory goes



Full fine-tuning carries a heavy optimizer state for every weight. LoRA freezes the base and trains tiny adapters. QLoRA also shrinks the base to 4-bit and pages the optimizer to CPU RAM on a spike. Same idea, three points on the memory budget.

The money slide: read it across

method	trainable %	peak VRAM*	quality	when to use
Full SFT	100%	~5.9 GB	best ceiling	you can afford the compute
LoRA	~2%	~3.4 GB	≈ full	the default for most work
QLoRA	~2%	~2.3 GB	≈ full, tiny cost	when VRAM is the constraint

*Qwen3-0.6B, 60 steps, Colab T4, representative reference numbers; your run populates the real ones.

*A lower loss is **not** the same as good clinical answers. Whether the fine-tune actually worked is a measurement problem, and that deserves its own careful pass.*

PART 5 · NOTEBOOK 04

Deployment

A trained model on disk is not a product. Someone has to be able to call it.

Three ways to serve (what NB04 runs)

A · LOCAL IN-PROCESS

`model.generate()` in your own process. Zero network, full privacy, you hold the GPU. **This is what an Organon skill does.**

B · SELF-HOSTED SERVER

`vllm serve` or `TGI`: continuous batching and an OpenAI-compatible API. Scales to many users on your own GPU; data stays on your infra.

C · ON-DEVICE (GGUF)

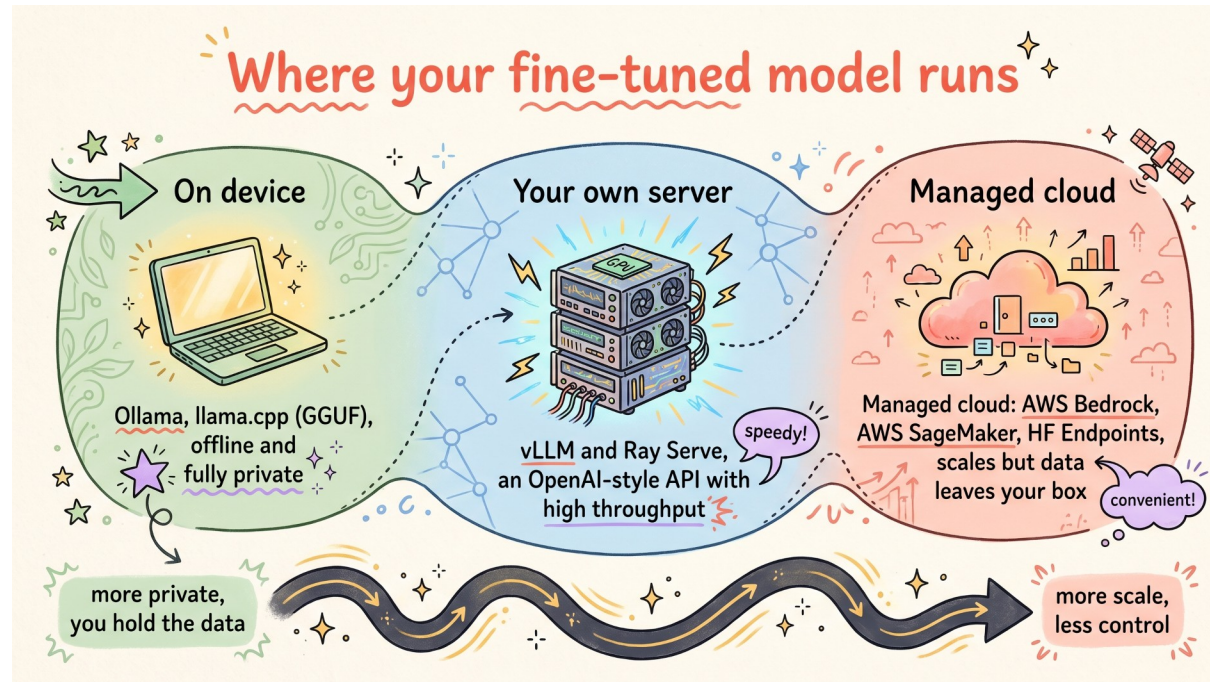
Convert to GGUF, run with `llama.cpp` or `Ollama` on CPU or Metal. Offline; data never leaves the device.

pattern	latency	cost	privacy	scale
A local	low	free	total	no
B server	low, batched	your GPU	your infra	yes
C on-device	medium	free	total	no

For clinical and scientific data, privacy often decides before latency or cost.

A small model you can run on-device is frequently the *right* answer, even if a giant hosted model benchmarks higher.

From your laptop to the cloud



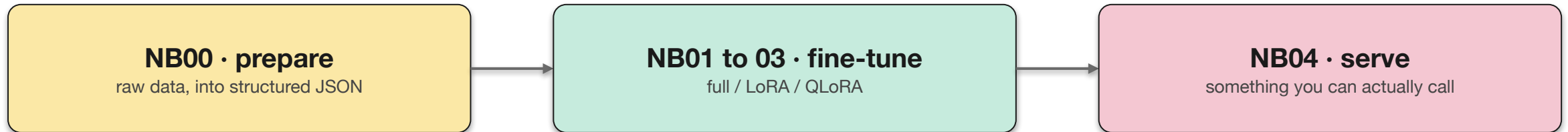
The same fine-tuned model can run three places. Left to right you trade privacy and control for scale: keep it on-device when the data must not move, reach for a managed cloud when you need to serve the world.

Pick your serving stack

tool	runs on	best for	scales	data
transformers .generate()	your process	quick tests, an Organon skill	no	stays local
Ollama / llama.cpp (GGUF)	your device	offline, private, laptop or edge	no	stays local
vLLM	your GPU server	high-throughput OpenAI-style API	1 node, batched	your infra
Ray Serve	your cluster	many models, autoscaling	across nodes	your infra
HF Inference Endpoints	HF cloud	one-click deploy a Hub model	managed	HF cloud
AWS SageMaker	your AWS account	enterprise MLOps, your own model	managed	your AWS
AWS Bedrock	serverless API	foundation models, no infra to run	managed	AWS

Colour = where your data lives: **green** on your machine (most private), **blue** your own server or cluster, **pink** a managed cloud. NB04 covers the green and blue rows hands-on; the pink rows are the same model, deployed by someone else's platform.

What you did today



WHAT YOU CAN NOW DO

Take a base model and your own dataset, and produce a model that answers in the format you need. Full, LoRA, or QLoRA, depending on the VRAM you have.

THE QUESTION THAT REMAINS

Is it any good? A lower loss does not answer that. You find out by measuring the model against ground truth, and that is a craft of its own.

CLOSE · 5 MIN

Run it yourself

Five notebooks back every idea today. The links land in chat: fork them, break them, make them yours.

The five notebooks

- [00 · Dataset prep](#): raw data → structured chat-template JSON
- [01 · Full SFT](#): update 100% of weights → strongest, heaviest
- [02 · LoRA](#): freeze the base → train small adapters
- [03 · QLoRA](#): 4-bit base + adapters → fits one GPU
- [04 · Deploy & inference](#): serve it → call it like an API

► [Open all in Colab · github.com/krmde/agent-ai-for-scientists-workshop → week-04-post-training/notebooks](#)

Click any notebook title to open it in Colab. Same code across all three: one flag flips full into LoRA into QLoRA.

Questions?

Open the notebooks, 00 → 04, and fine-tune your first model.